1 Cel ćwiczenia

Zapoznanie się ze statystykami opisowymi i metodami wizualizacji danych oraz ich implementacją w Pythonie.

2 Treść zadania

Dokonaj opisu statystycznego i wizualizacji danych charakteryzujących samochody.

3 Wykonanie ćwiczeń

3.1 Zadanie 1

Polecenie: Wczytaj i zmodyfikuj zbiór danych:

```
1 import pandas as pd
2 import numpy as np
3
4 data = pd.read_csv('carbig.csv')
5 np.random.seed(int(3.8 * 2025))
6
7 # Replace NaN
8 columns = ['Acceleration', 'Displacement', 'Horsepower', 'Weight', 'MPG']
9 for col in columns:
10   data[col].fillna(data[col].mode()[0], inplace=True)
11
12 # Select columns
13 x = data[columns].values
14
15 # Add random noise (1% variation)
16 noise_multiplier = 1 + (np.random.rand(*x.shape) * 2 - 1) / 100
17 x = x * noise_multiplier
```

Objaśnij powyższe polecenia. Podejrzyj i opisz zmienne.

Rozwiązanie:
Objaśnienie poleceń:
Linijka 4. wczytuje dane dotyczące samochodów.
Linijka 5. ustawia ziarno generatora liczb pseudolosowych.
Linijka 8. definiuje analizowane kolumny.
Linijki 9-10 w pętli podmieniają brakujące wartości na najczęściej występującą w danej kolumnie wartość.
Linijka 13. tworzy macierz danych.
Linijki 16-17 dodają losowy szum.

Opisanie zmiennych:
`data` – DataFrame zawierający dane wczytane z pliku.
`columns` – lista nazw wybranych kolumn, które będą brane pod uwagę.
`x` – tablica zawierająca wybrane kolumny.
`noise_multiplier` – tablica losowych wartości służących do dodania szumu do danych.

3.2 Zadanie 2

Polecenie: Wyznacz wartości minimalne i maksymalne oraz miary średnie dla każdej zmiennej numerycznej (kolumny macierzy x).

Rozwiązanie:

```python
df_x = pd.DataFrame(x, columns=columns)
descriptive_stats = df_x.describe().loc[['min', 'max', 'mean']]
descriptive_stats.loc['median'] = df_x.median()

unique_counts = df_x.nunique()
all_unique = unique_counts == len(df_x)

mode = df_x.mode()
for col in df_x.columns:
    if all_unique[col]:
        descriptive_stats.loc['mode', col] = np.nan
    else:
        descriptive_stats.loc['mode', col] = mode[col].iloc[0]
print(descriptive_stats)
```

```
        Acceleration  Displacement  Horsepower       Weight        MPG
min         7.951296     67.511576   45.949766  1599.222153   9.022294
max        25.011605    458.562755  228.526239  5134.525188  46.228956
mean       15.517602    194.770564  105.754828  2978.832303  23.315650
median     15.474616    149.656076   95.037314  2822.988676  22.392204
mode             NaN           NaN         NaN          NaN        NaN
```

Powyższy kod wyznacza wymagane wartości dla każdej kolumny. Pod nim znajdują się uzyskane wyniki.
Funkcja `pandas.DataFrame.describe` zwraca informacje o najmniejszej i największej wartości danej kolumny oraz jej średnią arytmetyczną.
Ta funkcja nie zwraca informacji o medianie, więc w linijce niżej dodaję ją oddzielnie do moich wyników.
W przypadku dominanty chcę najpierw sprawdzić, czy jakiekolwiek wartości w kolumnach się powtarzają – jeśli wszystkie są unikalne, to wszystkie występują tyle samo razy w zbiorze. Taki przypadek oznaczam poprzez wpisanie `NaN` jako dominanty.

3.3 Zadanie 3

Polecenie: Wyznacz rozstęp, kwartyle, rozstęp międzykwartylowy, odchylenie ćwiartkowe i typowy obszar zmienności dla każdej zmiennej numerycznej. Utwórz wykresy pudełkowe dla wszystkich zmiennych numerycznych (`boxplot`).

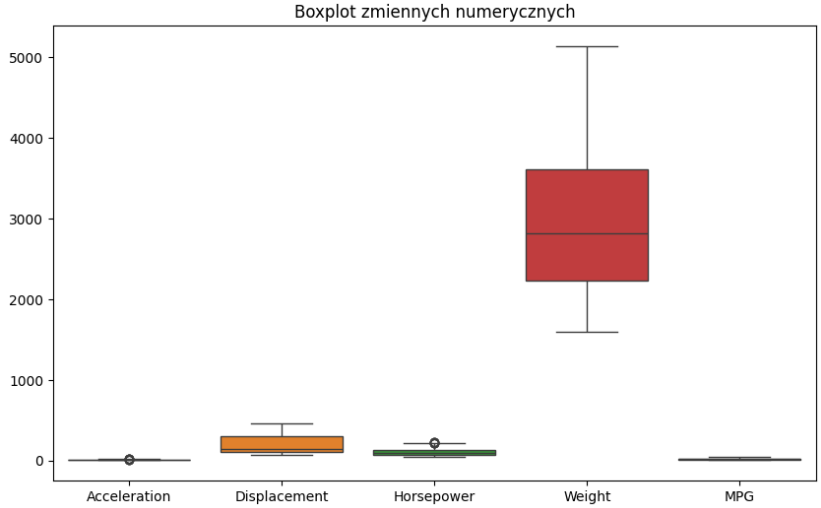
Rozwiązanie:

```python
results = pd.DataFrame({
    'Rozstęp': df_x.max() - df_x.min(),
    'Kwartyl pierwszy (Q1)': df_x.quantile(0.25),
    'Kwartyl drugi (Q2)': df_x.quantile(0.50),
    'Kwartyl trzeci (Q3)': df_x.quantile(0.75),
    'Rozstęp międzykwartylowy (IQR)': df_x.quantile(0.75) - df_x.quantile(0.25),
    'Odchylenie ćwiartkowe': (df_x.quantile(0.75) - df_x.quantile(0.25))/2,
    'Typowy obszar zmienności': (df_x.median() - df_x.quantile(0.25)).astype(str) +
    " ; " + (df_x.median() + df_x.quantile(0.25)).astype(str)
})

print("Statystyki opisowe:")
print(results)

plt.figure(figsize=(10, 6))
sns.boxplot(data=df_x)
plt.title("Boxplot zmiennych numerycznych")
plt.show()
```

Kod wyznaczający potrzebne wartości i generujący wykres pudełkowy.

```
Statystyki opisowe:
                     Rozstęp  Kwartyl pierwszy (Q1)  Kwartyl drugi (Q2)  \
Acceleration       17.060310              13.665012           15.474616
Displacement      391.051179             104.121180          149.656076
Horsepower        182.576473              75.707036           95.037314
Weight           3535.303035            2230.552126         2822.988676
MPG                37.206662              16.906480           22.392204

                 Kwartyl trzeci (Q3)  Rozstęp międzykwartylowy (IQR)  \
Acceleration               17.124958                        3.459945
Displacement              299.426760                      195.305581
Horsepower                131.570915                       55.863879
Weight                   3617.162062                     1386.609936
MPG                        28.955745                       12.049265

                 Odchylenie ćwiartkowe                 Typowy obszar zmienności
Acceleration                  1.729973  1.8096041626815609 ; 29.139628365555673
Displacement                 97.652790   45.53489604998565 ; 253.77725543693003
Horsepower                   27.931940   19.33027755779672 ; 170.74434996854905
Weight                      693.304968    592.4365501089683 ; 5053.540801735413
MPG                           6.024632    5.48572443589191 ; 39.298684362120625
```

Wyniki.



Wykres pudełkowy.

Powyższe dane, zwizualizowane na wykresie pudełkowym, pozwalają zauważyć, że rozpatrywane samochody są najbardziej zróżnicowane pod względem wagi, a najmniej pod względem przyspieszenia. Nie znaczy to jednak, że różnice w przyspieszeniu są znikome – dane pokazują spore zróżnicowanie w każdej z badanych cech.

3.4 Zadanie 4

Polecenie: Wyznacz miary rozrzutu dla każdej zmiennej numerycznej.

Rozwiązanie:

```
rozrzut = pd.DataFrame(index=columns)
rozrzut['Odchylenie standardowe'] = df_x.std()
rozrzut['Wariancja'] = df_x.var()
rozrzut['Współczynnik zmienności (%)'] = (rozrzut['Odchylenie standardowe'] / df_x.mean()) * 100

print("Miary rozrzutu dla każdej zmiennej numerycznej:")
print(rozrzut)
```

```
Miary rozrzutu dla każdej zmiennej numerycznej:
                  Odchylenie standardowe      Wariancja  \
Acceleration                    2.803189       7.857867
Displacement                  104.858771   10995.361795
Horsepower                     38.889055    1512.358609
Weight                        848.499340  719951.129920
MPG                             7.882789      62.138358

                  Współczynnik zmienności (%)
Acceleration                        18.064574
Displacement                        53.837073
Horsepower                          36.772841
Weight                              28.484294
MPG                                 33.809003
```

Powyższe wyniki pozwalają wyciągnąć następujące wnioski:

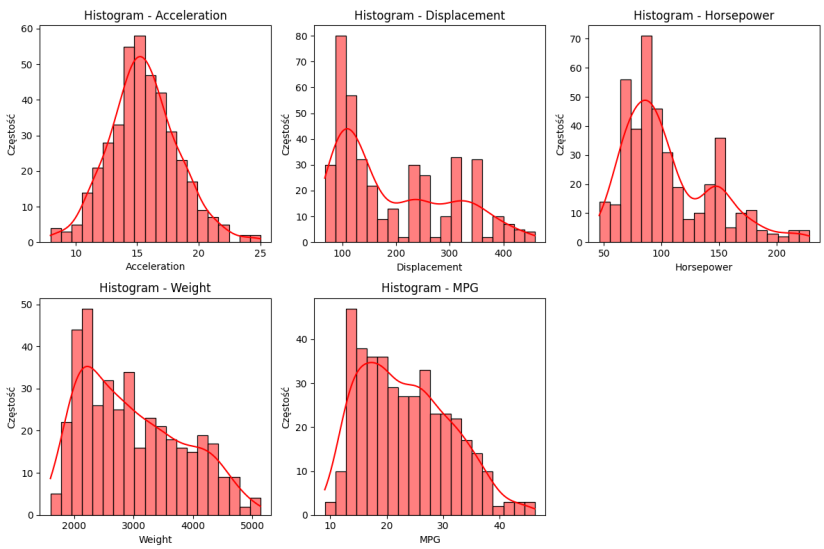
- Stosunkowo małe zróżnicowanie przyspieszenia (acceleration) – wartości nie odbiegają drastycznie od średniej.
- Bardzo zróżnicowanie pojemności silnika (displacement), co oznacza, że w zbiorze są zarówno małe, jak i bardzo duże silniki. Największy współczynnik zmienności.
- Dość zróżnicowana moc silnika (horsepower), ale mniej niż pojemność silnika.
- Duże różnice w masie pojazdów, ale mniej ekstremalne niż w przypadku pojemności silnika (większa wariancja i odchylenie standardowe, ale mniejszy współczynnik zmienności).
- Dość duże różnice w zużyciu paliwa (MPG - Miles per Gallon).

3.5 Zadanie 5

Polecenie: Utwórz histogramy dla każdej zmiennej numerycznej.

Rozwiązanie:

```python
plt.figure(figsize=(12, 8))

for i, col in enumerate(columns):
    plt.subplot(2, 3, i + 1)
    sns.histplot(df_x[col], bins=20, kde=True, color='red')
    plt.title(f'Histogram - {col}')
    plt.xlabel(col)
    plt.ylabel('Częstość')

plt.tight_layout()
plt.show()
```



Mogę wyciągnąć następujące wnioski z poszczególnych histogramów:

- **Acceleration** – rozkład zbliżony do rozkładu normalnego, tylko lekko asymetryczny w prawo, co oznacza, że większość samochodów ma podobne przyspieszenie, a skrajne wartości są rzadkie.
- **Displacement** – silna skośność w prawo, co wskazuje na to, że większość samochodów ma stosunkowo niską pojemność silnika, a tylko pojedyncze znacząco wiekszą. Oprócz tego widać kilka maksimów lokalnych, co może wskazywać na istnienie różnych grup samochodów i zależność między przynależnością do danej grupy a pojemnością silnika.
- **Horsepower** – podobnie jak przy pojemności silnika histogram skośny w prawo, co oznacza, że większość samochodów ma niską moc silnika.

Widoczne są dwa maksima, co może wskazywać na dwie grupy o różnej typowej mocy silnika.

- **Weight** – też skośny w prawo, ale bez widocznych maksimów, co pozwala wnioskować, że większość samochodów jest lekkich, ale istnieje też znacząca liczba cięższych modeli.
- **MPG** – większość samochodów ma niską wartość MPG, czyli wysokie spalanie, ale istnieją też bardziej oszczędne modele. Tak jak w przypadku wagi, nie widać wyróżniających poszczególne grupy maksimów.

3.6 Zadanie 6

Polecenie: Wyznacz współczynniki asymetrii (skew) oraz kurtozy (kurtosis) dla każdej zmiennej numerycznej.

Rozwiązanie:

```python
results = pd.DataFrame({
    'Kurtoza': df_x.kurtosis(),
    'Asymetria': df_x.skew()
})

print("Statystyki kurtozy i asymetrii:")
print(results)
```

```
Statystyki kurtozy i asymetrii:
               Kurtoza  Asymetria
Acceleration  0.402953   0.233809
Displacement -0.807774   0.695610
Horsepower    0.420389   0.986039
Weight       -0.817684   0.508819
MPG          -0.532852   0.469194
```

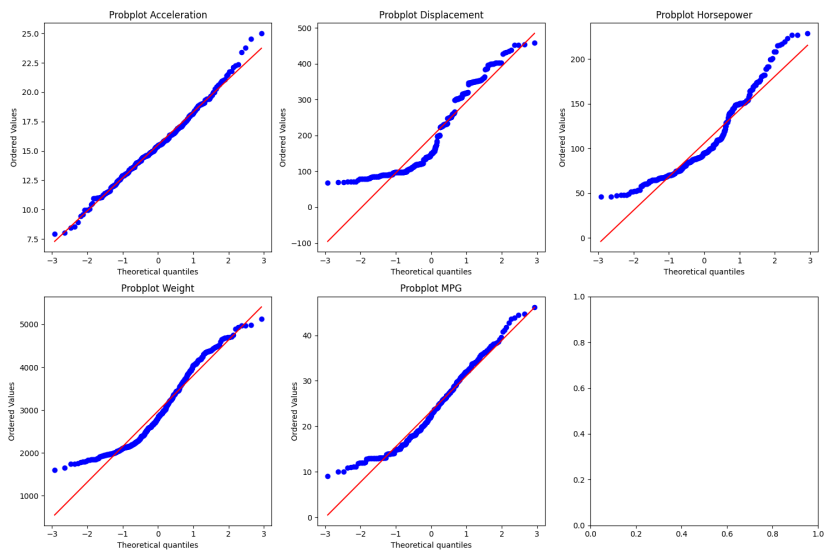
Powyższe wyniki potwierdzają wnioski z histogramów: przyspieszenie jest najbliższe rozkładowi normalnemu (asymetria najbliższa zeru), wszystkie rozkłady są skośne w prawo (wszystkie wartości asymetrii są dodatnie), a wartości pojemności silnika, wagi i MPG są bardziej rozproszone (nieskupione wokół średniej) niż w przypadku mocy silnika i przyspieszenia, co pokazuje ujemna kurtoza.

3.7 Zadanie 7

Polecenie: Utwórz wykres `probplot` dla każdej zmiennej numerycznej. Wyjaśnij co on obrazuje.

Rozwiązanie:

```
fig, axes = plt.subplots(2, 3, figsize=(15, 10))
axes = axes.flatten()
for i, col in enumerate(columns):
    stats.probplot(df_x[col], dist="norm", plot=axes[i])
    axes[i].set_title(f'Probplot {col}')
plt.tight_layout()
plt.show()
```



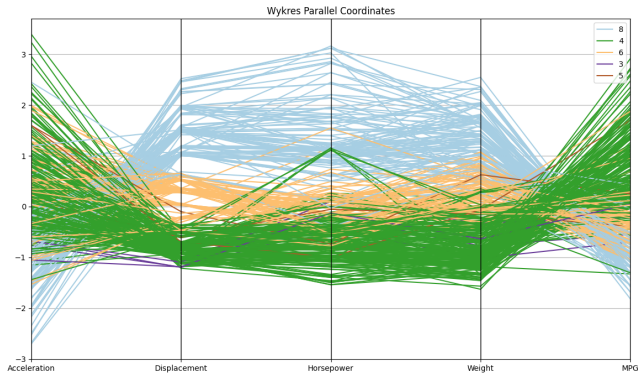
Wykresy pozwalają ocenić, czy zmienne mają rozkład normalny. Na osi X mamy kwantyle teoretyczne (czyli wartości oczekiwane, gdyby dane miały rozkład normalny), a na osi Y – kwantyle empiryczne (czyli rzeczywiste wartości naszych danych). Jeśli punkty układają się wzdłuż czerwonej linii mówimy o rozkładzie zbliżonym do normalnego. Wykresy wygenerowane dla moich danych pozwalają wnioskować, że Acceleration (przyspieszenie) jest najbliżej rozkładu normalnego. Displacement (pojemność silnika), Horsepower (moc silnika) i Weight (waga) mają widoczne odchylenia przy końcach, a i na środku przypasowanie nie jest idealne. W przypadku MPG początek jest bardzo odchylony, ale w środku punkty w większości pasują do linii. Te wykresy potwierdzają wnioski wyciągnięte z histogramów w zadaniu szóstym i współczynników asymetrii i kurtozy w zadaniu siódmym.

3.8 Zadanie 8

Polecenie: Utwórz i zinterpretuj wykres `parallel_coordinates`. Uwaga, przed wykonaniem wykresu dokonaj standaryzacji danych (`zscore`) i przekształć dane do postaci `DataFrame`.

Rozwiązanie:

```
scaler = StandardScaler()
x_scaled = scaler.fit_transform(x)

df_scaled = pd.DataFrame(x_scaled, columns=columns)

df_scaled['Cylinders'] = data['Cylinders']

plt.figure(figsize=(14, 8))
parallel_coordinates(df_scaled, 'Cylinders', colormap='Paired')
plt.title('Wykres Parallel Coordinates')
plt.show()
```



Każda linia na wykresie reprezentuje jeden samochód z zestawu danych. Oś pionowa to znormalizowane wartości cech (standaryzowane do podobnego zakresu). Linie są kolorowane według liczby cylindrów w silniku, zgodnie z legendą w prawym górnym rogu. Wykres pozwala analizować, jak Acceleration, Displacement, Horsepower, Weight i MPG zmieniają się między pojazdami o różnej liczbie cylindrów. Widoczne jest, że samochody z ośmioma cylindrami (jasnoniebieskie) mają największą pojemność i moc silnika i są najcięższe, ale mają najniższe przyspieszenie i duże spalanie (niskie MPG), podczas gdy samochody z czterema cylindrami (zielone) mają najmniejszą pojemność i moc silnika, są najlżejsze i najoszczędniejsze (mają najwyższe MPG). Pomiędzy tymi dwoma skrajnościami plasują się samochody z 6 cylindrami (beżowe). Samochodów z trzema lub pięcioma cylindrami jest mało w zbiorze danych, więc są słabo widoczne na wykresie i trudno jest wyciągnąć o nich jakiekolwiek wnioski.

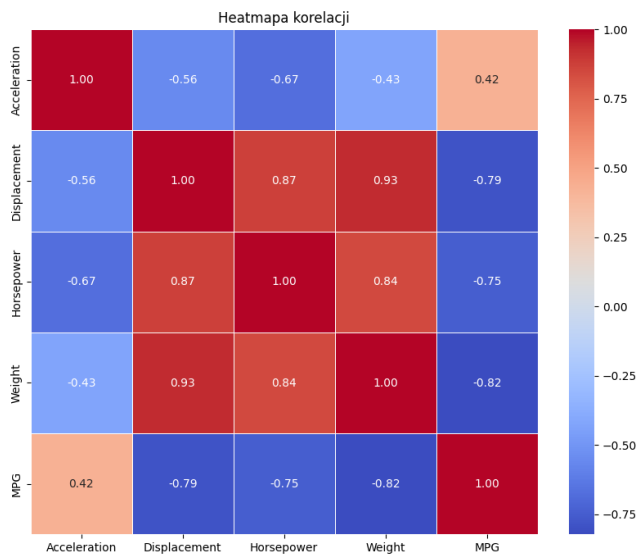
3.9 Zadanie 9

Polecenie: Wyznacz współczynniki korelacji liniowej dla każdej pary zmiennych numerycznych.

Rozwiązanie:

```python
print(df_x.corr())

plt.figure(figsize=(10, 8))
sns.heatmap(df_x.corr(), annot=True, cmap='coolwarm', fmt='.2f', linewidths=0.5)

plt.title('Heatmapa korelacji')
plt.show()
```

```
               Acceleration  Displacement  Horsepower    Weight       MPG
Acceleration       1.000000     -0.558426   -0.674948 -0.430690  0.424452
Displacement      -0.558426      1.000000    0.874240  0.931629 -0.793435
Horsepower        -0.674948      0.874240    1.000000  0.843876 -0.745291
Weight            -0.430690      0.931629    0.843876  1.000000 -0.822244
MPG                0.424452     -0.793435   -0.745291 -0.822244  1.000000
```



Z powyższych wyników zauważam następujące zależności:

- Acceleration i MPG (0.424452) – umiarkowana dodatnia korelacja, więc samochody z lepszym przyspieszeniem mają zwykle wyższą efektywność paliwową (MPG).
- Displacement i Horsepower (0.874240) – silna dodatnia korelacja, więc większa pojemność silnika oznacza więcej koni mechanicznych.
- Displacement i Weight (0.931629) – bardzo silna dodatnia korelacja, więc im większa pojemność silnika, tym cięższy samochód.
- Displacement i MPG (-0.793435) – bilna ujemna korelacja, więc większa pojemność silnika oznacza wyższe spalanie (mniejszy MPG).

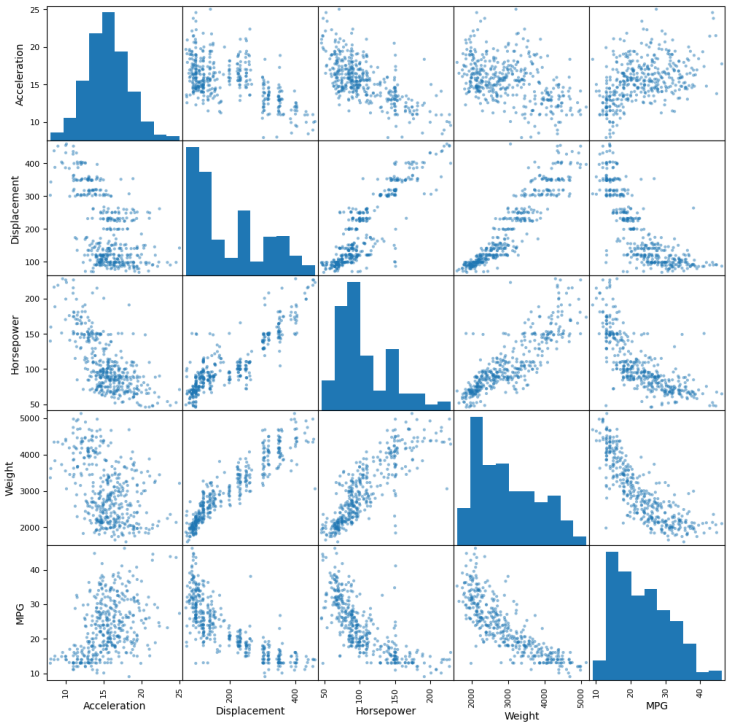
- Horsepower i MPG (-0.745291) – silna ujemna korelacja, więc im mocniejszy silnik, tym większe zużycie paliwa.
- Weight i MPG (-0.822244) – bardzo silna ujemna korelacja, więc cięższe samochody spalają więcej paliwa (niższy MPG).
- Acceleration i Displacement (-0.558426) – umiarkowana ujemna korelacja, więc większa pojemność silnika często wiąże się z mniejszym przyspieszeniem.
- Acceleration i Horsepower (-0.674948) – dość silna ujemna korelacja, więc większa moc nie zawsze oznacza lepsze przyspieszenie.

3.10 Zadanie 10

Polecenie: Utwórz wykres obrazujący współzależności pomiędzy każdą parą zmiennych numerycznych.

Rozwiązanie:

```
scatter_matrix(df_x, figsize=(12, 12), diagonal='hist')
plt.show()
```



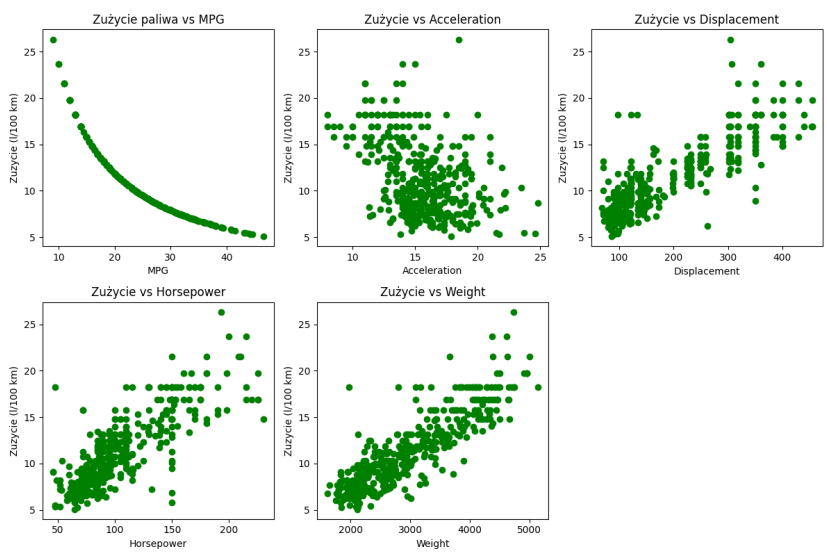
Na przekątnej widać histogramy każdej zmiennej, którym przyjrzałam się juz bliżej i zinterpretowałam w zadaniu piątym. Wykresy poniżej i powyżej przekątnej pokazują zależności między parami zmiennych. Można zauważyć, że skorelowane ze sobą w pewnym stopniu są wielkość silnika, moc silnika, waga i MPG. Przyspieszenie zdaje się nie być znacząco skorelowane z innymi zmiennymi.

3.11 Zadanie 11

Polecenie: Wyznacz dodatkową zmienną (Zuzycie) reprezentującą zużycie paliwa w litrach na 100 km (na podstawie zmiennej MPG). Przyjmij 1 mila = 1,6 km, 1 galon = 3,79 litra. Sporządź wykres zależności zużycia paliwa od MPG. Utwórz wykresy zależności Zuzycie od Acceleration, Displacement, Horsepower, Weight.

Rozwiązanie:

```
[14]  data['Zuzycie'] = 3.79 / (data['MPG'] * 1.6) * 100

      fig, axes = plt.subplots(2, 3, figsize=(12, 8))
      axes = axes.flatten()

      axes[0].scatter(data['MPG'], data['Zuzycie'], c="green")
      axes[0].set_xlabel("MPG")
      axes[0].set_ylabel("Zuzycie (l/100 km)")
      axes[0].set_title("Zużycie paliwa vs MPG")

      features = ['Acceleration', 'Displacement', 'Horsepower', 'Weight']
      for i, col in enumerate(features):
          axes[i+1].scatter(data[col], data['Zuzycie'], c="green")
          axes[i+1].set_xlabel(col)
          axes[i+1].set_ylabel("Zuzycie (l/100 km)")
          axes[i+1].set_title(f'Zużycie vs {col}')

      fig.delaxes(axes[-1])

      plt.tight_layout()
      plt.show()
```



Powyższe wykresy pozwalają wyciągnąć następujące wnioski na temat zależności między zużyciem paliwa a resztą cech:

- Zużycie paliwa i MPG (Miles Per Gallon) – wykres pokazuje wyraźną zależność odwrotną między zużyciem paliwa a MPG, to znaczy im wyższe MPG, tym niższe zużycie paliwa. Jest to zgodne z oczekiwaniami i logiczne.
- Zużycie paliwa i Przyspieszenie – w ykres nie pokazuje wyraźnej korelacji między zużyciem paliwa a przyspieszeniem. Punkty są szeroko rozrzucone, co sugeruje, że przyspieszenie samochodu nie ma bezpośredniego wpływu na zużycie paliwa.
- Zużycie paliwa i Wielkość silnika (Displacement) – istnieje korelacja między zużyciem paliwa a wielkością silnika, widać, że im większy silnik, tym wyższe zużycie paliwa. Jest to zgodne z oczekiwaniami.
- Zużycie paliwa i Moc (Horsepower) – wykres pokazuje pewną korelację między zużyciem paliwa a mocą silnika. Samochody o wyższej mocy silnika zużywają więcej paliwa.
- Zużycie paliwa i Waga – istnieje korelacja pozytywna między zużyciem paliwa a wagą samochodu. Na wykresie widać, że cięższe samochody zużywają więcej paliwa.

3.12 Zadanie 12

Polecenie: Jakie jest średnie zużycie paliwa w dla samochodów z 3, 4, 5, 6 i 8 cylindrami?

Rozwiązanie:

```
cylinder_avg_consumption = data.groupby('Cylinders')['Zuzycie'].mean()
print(cylinder_avg_consumption)

Cylinders
3    11.659749
4     8.534553
5     9.167354
6    12.191568
8    16.436886
```

Największe średnie zużycie paliwa jest w samochodać ośmiocylindrowych, a najmniejsze w czterocylindrowych.

4 Wnioski końcowe

Wykonanie zadania udowodniło mi, że statystyki opisowe są fundamentalnym narzędziem w analizie danych, umożliwiającym zrozumienie zestawu danych i rzeczy, które one opisują. Wartości takie jak średnie, minima, maksima, odchylenia czy kwartyle pomagają uzyskać pełniejszy obraz opisywanego przez

dane zjawiska. Wnioskowanie jeszcze bardziej przyspiesza i ułatwia wizualizacja uzyskanych wyników. W tak pokazanych wynikach łatwiej jest dostrzec zależności, korelacje, anomalie i wzorce. I statystyka opisowa, i wizualizacja uzyskanych wyników, pozwalają zapoznać się ze strukturą danych bez potrzeby ich bezpośredniej, dokładnej analizy rekord po rekordzie.
Python oferuje wiele narzędzi i bibliotek do analizy i wizualizacji danych. Właściwie każdą potrzebną wartość można uzyskać za pomocą istniejącej już w jakiejś bibliotece funkcji. Do tych bibliotek należą np.pandas (do obróbki danych, obliczania statystyk opisowych), matplotlib i seaborn (do tworzenia wykresów, takich jak heatmapy, histogramy czy boxploty). Dzięki popularności języka Python w analizie danych łatwo jest znaleźć sposób na rozwiązanie problemu.